Parallel PCA Network Anomaly Detection — Technical Report

Nov 8, 2026 · @anurag

Abstract

We parallelised the PCA subspace method for network-wide traffic anomaly detection (Lakhina, Crovella and Diot, SIGCOMM 2004) with OpenMP. On 16 threads the whole pipeline runs 2.4× faster than the sequential version, and an injection experiment of 20,736 independent PCA fits runs 7.2× faster. Both are measured on a laptop Intel Core i5-13500H.

The method models link traffic as a low-dimensional normal subspace plus a residual. It raises an alarm when the residual energy (SPE) of a time bin exceeds the Q-statistic threshold. We implemented the detection step twice in C++17, once sequentially and once with OpenMP, and every OpenMP run produced results bitwise identical to the sequential one.

On held-out synthetic traffic (60 links, 144 OD flows, 10-minute bins) detection reached precision 1.00, recall 0.77 and F1 0.87. A Roofline analysis shows that the covariance kernel is compute-bound, while mean and centring are memory-bound. End-to-end speedup is capped by Eigen's serial eigensolver, as Amdahl's law predicts.

1. Application

The application is detecting volume anomalies across a whole network from per-link byte counts: DDoS floods, outages and sudden traffic shifts. Operators already collect these counts with SNMP, so the method needs no packet inspection.

An origin–destination (OD) flow is all traffic entering the network at one point of presence (PoP) and leaving at another. An anomaly in one OD flow adds traffic to every link on its path. On any single link the spike is small next to the normal daily cycle, which makes it hard to see. Lakhina et al. showed that looking at all links together exposes it. Normal traffic on 41–49 backbone links is explained by 3–4 shared temporal patterns, and an anomaly breaks that shared structure.

The paper diagnoses anomalies in three steps: detect, identify the OD flow, quantify the bytes. This project implements the detection step. The HPC motif is dense linear algebra and the parallel framework is OpenMP.

Data. All results use a synthetic network built the way the paper built its link data, $y = A\,x$: link traffic is OD-flow traffic routed through a routing matrix A.

Property	Value
Topology	12 PoPs, 18 edges (ring plus 6 random chords)
Links (m)	60: 36 directed internal + 12 ingress + 12 egress
OD flows	144, gravity-model means, diurnal and weekly profiles, per-PoP time-zone shift, log-normal noise
Time	2 weeks × 1,008 bins of 10 minutes; week 1 trains, week 2 tests
Labelled anomalies per week	10 single-flow spikes, 3 outages, 3 multi-flow DDoS events (4–6 origins → 1 destination)

The benchmarks use larger seeded matrices (up to 65,536 × 2,048) with the same daily, half-daily and weekly structure. They measure speed, not detection quality. Real datasets (Abilene traffic matrices, CICIDS2017) were planned but left out to meet the one-day deadline.

2. Model

The model splits each time snapshot of link traffic into a normal part, which lies in an r-dimensional PCA subspace, and a residual. Detection only ever looks at the residual.

Measurement matrix. Y is t × m: rows are time bins, columns are links, and y is one row. This is the paper's orientation. In memory, Eigen stores Y column-major, so each link's time series is contiguous; physically that is the link × time layout of the project registration.

Steps.

1. Centre every column: subtract the link mean μ_j. Columns are not scaled, because the paper wants PCA to capture true byte variance.
2. Form the covariance matrix and decompose it. The eigenvectors v_i are the principal axes, sorted by decreasing eigenvalue λ_i.

$$\Sigma = (1 / (t - 1)) \cdot Y^T Y = V \Lambda V^T$$

3. Choose r with the paper's 3σ rule. Walk the axes in order and project the data onto each, $u_i = Y v_i$. The first axis whose projection deviates more than 3 standard deviations from its mean, and every axis after it, form the anomalous subspace. The axes before it form the normal subspace, $P = [v_1 \dots v_r]$. Fixed r and a variance-fraction rule are also available.
4. Split each snapshot and score it by its squared prediction error (SPE):

$$\hat{y} = P P^T y, \quad \tilde{y} = (I - P P^T) y, \quad SPE = \|\tilde{y}\|^2$$

5. Raise an alarm when SPE exceeds the Jackson–Mudholkar Q-statistic threshold at confidence $1 - \alpha$ ($\alpha = 0.001$). Here $c\alpha$ is the standard normal quantile:

$$\delta^2\alpha = \varphi_1 \cdot \left[c\alpha \cdot \sqrt{2\varphi_2 h_0^2} / \varphi_1 + 1 + \varphi_2 h_0 (h_0 - 1) / \varphi_1^2 \right] \wedge (1 / h_0)$$

where $h_0 = 1 - 2\varphi_1\varphi_3 / (3\varphi_2^2)$ and $\varphi_i = \Sigma \lambda_j^i$ summed over the residual eigenvalues $j = r+1 \ldots m$, for $i = 1, 2, 3$.

The threshold is analytic: no labels are used to set it. Labels are read only after detection, to compute precision, recall, F1 and false-positive rate.

Decisions where the paper is ambiguous.

- **Scale of λ.** The paper writes $\lambda_j = \|Y v_j\|^2$, which is about t times a covariance eigenvalue. SPE is a single-row quantity, so we use $\|Y v_j\|^2/(t-1)$. A unit test shows the literal reading produces a threshold that nothing ever exceeds.
- **$h_0 \leqslant 0$.** For very skewed residual spectra the formula above is undefined. We then fall back to Box's approximation, SPE $\approx g \cdot \chi^2_h$ with $g = \varphi_2/\varphi_1$ and $h = \varphi_1^2/\varphi_2$. Our synthetic data needed this fallback, because the first axis holds 95% of the variance.
- **r bounds.** r is clamped to $[1, m-1]$, and axes with zero numerical variance are never tested by the 3σ rule.
- **Fit and apply.** Both the paper's in-sample mode and a train/test mode are supported. In train/test mode, μ, P and δ^2 come from the training week only.

3. Implementation and correctness

The sequential and OpenMP versions run the same algorithm and differ only in how work is shared between threads. Every check we ran confirms they agree.

Structure. The pipeline is written once, as a C++ template over a kernel backend. The heavy kernels exist twice with identical signatures:

- `pca::seq` is compiled without the OpenMP runtime (only `-fopenmp-simd` for vectorisation hints);
- `pca::omp` is the parallel version.

The arithmetic inside each tile or row block lives in one shared header, so both versions perform the same floating-point operations in the same order. Eigen 3.4 supplies the matrix type and the symmetric eigensolver, built with `EIGEN_DONT_PARALLELIZE` so that every thread in the program is one we created. Executables: `pca_seq`, `pca_omp`, `injection_study`, `bench` and `pca_tests`.

Correctness checks.

Check	Result
C++ unit tests: hand-computed covariance, odd matrix sizes, rank-1 data, injected spike, invalid input, 1e9 offset	1,208 checks, 0 failures
Independent NumPy implementation (SVD route), 4 cases × 2 executables	eigenvalues within 2.6e-15 · λ_1, SPE within 2.9e-12 relative, identical r and alarms
OpenMP vs sequential, every benchmark configuration	31 of 31 bitwise-identical SPE, same r, same alarms
Q-statistic false-alarm rate, Gaussian data, t = 200,000	0.967% at α = 1%, 0.105% at α = 0.1%

Detection quality. Fitted on week 1 and tested on held-out week 2, with α = 0.001 and r chosen by the 3σ rule:

Setting	r	Precision	Recall	F1	False-positive rate	Events found
Train week 1, test week 2	2	1.000	0.773	0.872	0.000	spikes 10/10, DDoS 3/3, outages 0/3
In-sample on week 2 (paper's setting)	1	1.000	0.636	0.778	0.000	spikes 7/10, DDoS 3/3, outages 0/3

- **Outages are missed.** They remove only 9–22 MB, against 65–130 MB for the spikes, so they project too little onto the residual subspace.
- **In-sample fitting is weaker.** The anomalies pull the principal axes towards themselves, an effect the paper also notes.
- **False alarms exceed the nominal rate on clean data.** On the anomaly-free week the rate was 2.7%, against α = 0.1%, because the synthetic residuals are not Gaussian.

4. Parallelism

Each kernel is parallelised over independent outputs, so no two threads ever write the same value. That design needs no locks or atomics, and it keeps the OpenMP results bitwise identical to the sequential ones.

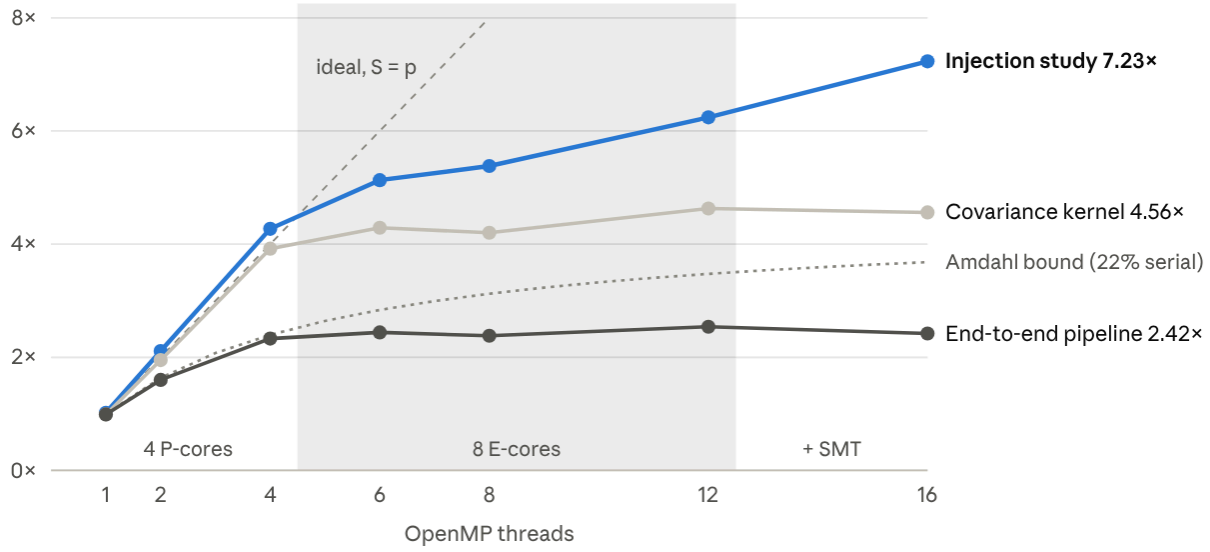
Kernel	Cost	Work split between threads	Schedule and construct
Column means	O(tm)	one link per iteration	`parallel for`, `static`
Centring	O(tm)	one link per iteration	`parallel for`, `static`
Covariance Y^TY	O(tm²)	64 × 64 output tiles of the upper triangle, rows streamed in 512-row chunks	`parallel for`, `dynamic,1`
Eigendecomposition	O(m³)	not parallelised: Eigen, serial in both builds	—
3σ test statistics	O(t) per axis	rows of the projection	`reduction(+)`, `reduction(max)`
Projection + residual + SPE (fused)	O(tmr)	blocks of time rows	`parallel for`, `static`, thread-private workspace
Threshold and alarm count	O(t)	time bins	`reduction(+)` on an integer
Injection study	one full PCA fit per experiment	experiments (outer loop only)	`parallel for`, `dynamic,1`

- **Why split by output tile.** Splitting the covariance sum over time instead would need a private m × m accumulator per thread (16 × 8 MB at m = 1,024) and a merge step. Its result would also depend on the thread count.
- **Why `dynamic` scheduling.** Diagonal tiles do half the work of the others, and the CPU mixes fast P-cores with slower E-cores. Handing out one tile at a time lets the fast cores take more.
- **Cache blocking.** A tile's two column panels (2 × 64 × 512 × 8 B = 512 KiB) fit in L2. One load of a value feeds four dot products (register blocking).
- **Kernel fusion.** Scoring computes the projection, reconstruction, residual and SPE in one pass and never writes the reconstructed t × m matrix.

- **No nested parallelism or oversubscription.** Eigen's own threading is disabled. In the injection study only the outer loop is parallel, and each experiment runs the sequential kernels.

Independent fits reach 7.2×; the pipeline stops at 2.4× behind the serial eigensolver

Speedup over the sequential build, t = 32,768 bins, m = 1,024 links, i5-13500H



benchmarks/results/summary_pipeline.csv and summary_injection.csv · median of 3–5 runs per point

Covariance scales almost linearly on the four P-cores (3.92× at 4 threads), then flattens. Each E-core delivers 14.3 GFLOP/s against 36.4 GFLOP/s for a P-core, so the machine holds about 7.1 P-core-equivalents rather than 12. The injection study has no serial part and keeps improving to 7.23×. With one thread, OpenMP matches the sequential build to within 1%, so the runtime adds almost no overhead.

At the paper's own scale (1,008 × 60) OpenMP does not pay: the fit and score take 0.76 ms sequentially and 2.3 ms on 8 threads, because starting and scheduling threads costs more than the work.

5. Bottleneck and Roofline analysis

The pipeline's real bottleneck is not a kernel but the serial eigensolver. Among the kernels, covariance is compute-bound, while means and centring are memory-bound. Scoring sits right at the ridge point.

The Roofline model bounds attainable performance by the compute peak or by memory bandwidth times arithmetic intensity (AI), whichever is lower:

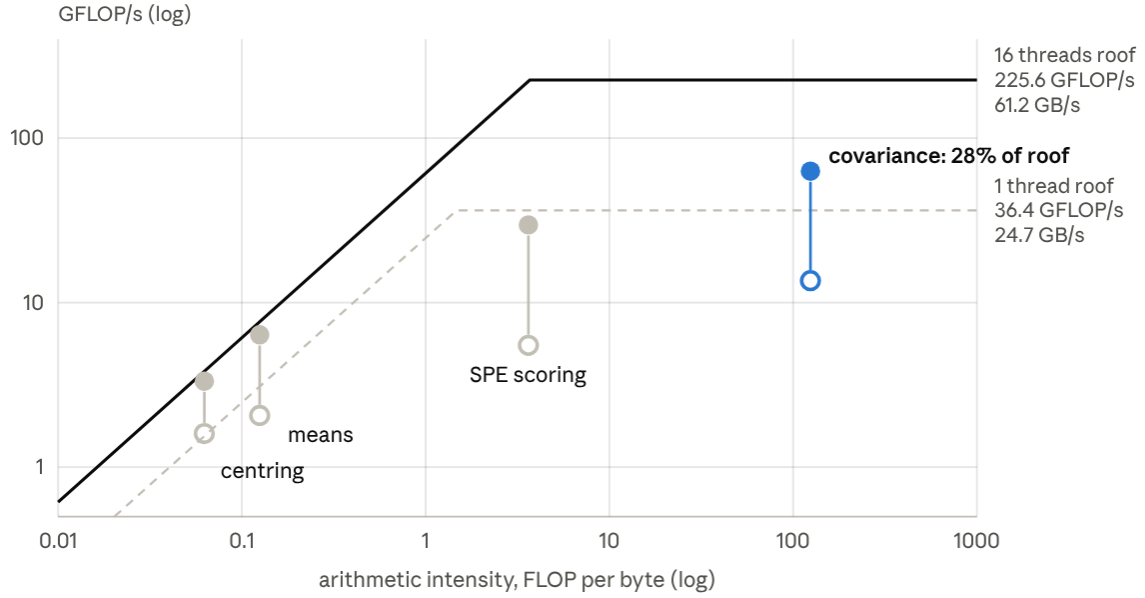
$P = \min(P_{peak}, B \times AI)$, AI = FLOPs / bytes moved

Ceiling	1 thread	Best, all threads	How obtained
FP64 peak	36.4 GFLOP/s	225.6 GFLOP/s (16 threads)	measured: register-resident FMA loop
Memory bandwidth	24.7 GB/s	61.2 GB/s (16 threads)	measured: STREAM triad, 32 B per element including write-allocate
Ridge point	1.47 FLOP/byte	3.69 FLOP/byte	peak divided by bandwidth
Theoretical FP64 peak	—	288 GFLOP/s	assumed, not measured: 4 P-cores × 16 FLOP/cycle × 2.6 GHz + 8 E-cores × 8 FLOP/cycle × 1.9 GHz

The firmware caps the clocks at 2.6 GHz (P-cores) and 1.9 GHz (E-cores). The theoretical DRAM bandwidth is not given, because the memory type cannot be read without root access.

Covariance is compute-bound yet uses 28% of its roof; means and centring sit on the memory roof

Hollow = sequential, filled = OpenMP 12 threads · t = 32,768, m = 1,024 · roofs measured on this laptop



benchmarks/results/roofline_table.csv, peak.csv, stream.csv · kernel FLOPs and bytes counted analytically, times measured

Kernel	FLOPs	Compulsory bytes	Bound	Sequential, % of roof	OpenMP 12 threads, % of roof
Means	tm	8tm	memory	67%	83%

Kernel	FLOPs	Compulsory bytes	Bound	Sequential, % of roof	OpenMP 12 threads, % of roof
Centring	tm	16tm (read + write)	memory	103%	87%
Covariance	t·m(m+1)	8tm + 8m²	compute	37%	28%
SPE scoring	(4r+5)·tm, r = 6	8tm	memory, at the ridge	15%	13%

- **Means and centring** move 8 or 16 bytes for every FLOP (AI 0.125 and 0.0625) and already run near the bandwidth roof. More threads help only until the memory bus saturates. Centring's 103% is within the noise between a best-of-10 microbenchmark and a median kernel time.
- **Covariance** has AI ≈ m/8 = 124 FLOP/byte, 34 times to the right of the ridge. Cache blocking is what makes that intensity real: without tiles every dot product re-reads two columns, and the AI drops to 0.125.
- **Why covariance reaches only 28–37%.** Its 1 × 4 register block needs 5 loads per 4 vector FMAs, so the load ports, not the FMA units, set the pace. Tuned BLAS kernels use 2-D register blocks (for example 4 × 12) and packed panels to reach 80–90%.
- **Scoring** reaches only 13–15% of its roof. Its inner loops run over just r = 6 columns, and it makes two passes over each row block.

Amdahl's law: the serial eigensolver

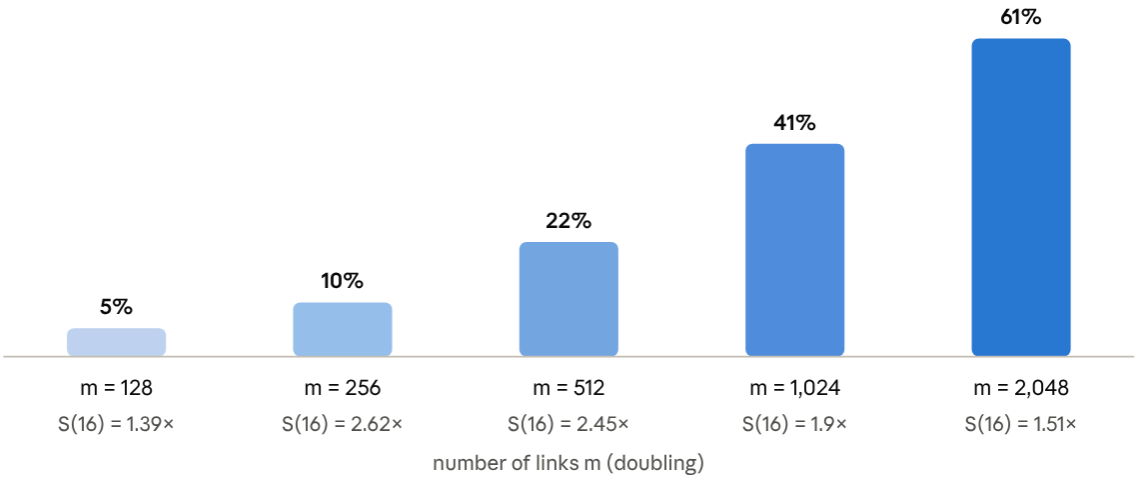
Covariance costs O(tm²) and runs in parallel; the eigendecomposition costs O(m³) and runs serially. With a serial fraction f, speedup on p threads is bounded by:

S(p) ≤ 1 / (f + (1 − f) / p)

At t = 32,768 and m = 1,024, f = 0.22, so the bound at 16 threads is 3.68×. We measured 2.42×.

The serial eigensolver grows from 5% to 61% of run time, so pipeline speedup falls to 1.5×

Eigensolver share of sequential run time, t = 16,384 bins · below each bar: end-to-end speedup on 16 threads



benchmarks/results/summary_pipeline.csv · m sweep at t = 16,384, median of 3 runs

At m = 2,048 the eigensolver takes 8.3 s of a 13.8 s sequential run. End-to-end speedup is then held to 1.5× even though the covariance kernel itself runs 5.0× faster. Growing t instead of m shrinks the serial share: at t = 65,536 and m = 512 the pipeline reaches 3.23×.

Hybrid cores add a second limit. Statically scheduled kernels finish only when their slowest thread does. Above 4 threads that slowest thread runs on an E-core, so the scoring kernel dips at 6–8 threads. The FMA test shows the same effect: 6 threads delivered 85.6 GFLOP/s, exactly 6 × 14.3 GFLOP/s, the E-core rate.

6. Limitations, future work and conclusion

The parallel implementation is correct and measurably faster, but the evidence has clear limits.

Limitations.

- **Detection only.** The paper's identification and quantification steps are not implemented.
- **Synthetic data only.** Detection numbers describe our generator, not a real network. The Abilene traffic matrices and CICIDS2017 were left out to meet the deadline.
- **The Gaussian assumption fails here.** On clean data the false-alarm rate was 2.7% against a nominal 0.1%, and $h_0 < 0$ forced the Box χ^2 fallback.
- **The 3σ rule chose r = 1–2,** not the paper's r = 4, and anomalies in the training data affect it.

- **One laptop.** It has a hybrid P/E-core CPU with firmware-capped clocks. Speedups will differ on a homogeneous server CPU. Windows builds are supported in code but untested.

Future work, in order of expected gain.

1. A parallel top-k eigensolver (block subspace iteration) to remove the Amdahl bottleneck. The 3σ rule needs only the leading eigenpairs.
2. A 2-D register-blocked covariance micro-kernel, benchmarked against BLAS `dsyrk`.
3. Real data: Abilene traffic matrices first, since their routing matrix allows the paper's full construction.
4. The identification step, which tests every OD flow per alarm and is itself an easily parallel workload.

Conclusion. OpenMP sped up the dense linear algebra kernels by up to 5.4× and the independent-fit workload by 7.2× on 16 threads, with results bitwise identical to the sequential build. The Roofline separates compute-bound covariance from memory-bound preprocessing. Amdahl's law with the serial $O(m^3)$ eigensolver explains why end-to-end speedup stays at 1.5–3.2×. Parallelising the eigensolver is therefore the next step that matters.